

Assignment01 TapuwaKabaira

Tapuwa Kabaira

September 2026

1 Problem 1: Merge Overlapping Intervals

Goal: Given intervals $[start, end]$, combine every pair of overlapping intervals so that it returns a non-overlapping set.

Input: A list of intervals, and each interval contains a start and end value.

Output: A list containing the merged, non-overlapping intervals sorted by their starting values.

Assumptions/edge cases: The rule of thumb is if next start current end, merge the smaller start and larger end. Intervals that touch at an endpoint are treated as overlapping for this problem, so $[1, 3]$ and $[3, 5]$ are merged

i. Design the Solution

First, sort the intervals by their starting value, in ascending order. Then scan from left to right. Compare each interval with the last interval in the result. If they overlap, extend the result interval's end value. Otherwise, add a new interval. Make sure not to lose or duplicate the values.

ii. Pseudocode

Input: An array of intervals, Intervals

Output: An array of non-overlapping intervals

```
1: if Intervals is empty then
2:   return empty list
3: end if
4: Sort Intervals by starting value
5: Merged <- empty list
6: Current <- first interval
7: for each Next interval starting from the second interval do
8:   if Next.start <= Current.end then
9:     Current.end <- max(Current.end, Next.end)
10:  else
11:    Add Current to Merged
12:    Current <- Next
```

```

13:     end if
14: end for
15: Add Current to Merged
16: return Merged

```

iii. Correctness

After sorting, every interval considered later starts at least as late as the previous intervals. The last interval in Result represents all overlapping intervals processed so far.

iv. Complexity

Sorting takes $O(n \log n)$ time and the scan takes $O(n)$ time. Space is $O(n)$ for the result.

v. Tests

- Test case : $[[1, 3], [2, 6], [8, 10], [9, 12]] \rightarrow [[1, 6], [8, 12]]$.
- Additional test: $[[1, 4], [4, 5]] \rightarrow [[1, 5]]$.



Figure 1: Problem 1 Test Case 1

2 Problem 2: Sort 0, 1, and 2

Goal: Sort an array containing only 0, 1, and 2 in place.

Input: An array containing only 0, 1, and 2.

Output: The same array sorted in place by category.

Required approaches: First use counting. Then use the one-pass Dutch National Flag algorithm without `std::sort`.

i. Design the Solution

The counting solution records how many 0s, 1s, and 2s occur, then overwrites the array in category order. The Dutch National Flag solution maintains three regions: 0s on the left, unknown values in the middle, and 2s on the right.

ii. Pseudocode – Counting

Input: An array of values, Values

Output: Values sorted in place

```
1: Count[0] <- 0
2: Count[1] <- 0
3: Count[2] <- 0
4: for each Value in Values do
5:     Count[Value] <- Count[Value] + 1
6: end for
7: Index <- 0
8: for Value <- 0 to 2 do
9:     for j <- 1 to Count[Value] do
10:         Values[Index] <- Value
11:         Index <- Index + 1
12:     end for
13: end for
14: return Values
```

iii. Pseudocode – Dutch National Flag

Algorithm Dutch National Flag

Input: An array of values, Values

Output: Values sorted in place

```
1: Low <- 0
2: Mid <- 0
3: High <- SizeOf(Values) - 1
4: while Mid <= High do
5:     if Values[Mid] = 0 then
6:         Swap Values[Low] and Values[Mid]
7:         Low <- Low + 1
8:         Mid <- Mid + 1
9:     else if Values[Mid] = 1 then
10:        Mid <- Mid + 1
11:    else
12:        Swap Values[Mid] and Values[High]
13:        High <- High - 1
14:    end if
15: end while
16: return Values
```

iv. Correctness

For counting, every input value contributes to exactly one count, and the over-write phase writes exactly those counts in the order 0, 1, 2.

v. Complexity

The algorithm takes $O(n)$ time because it makes one pass through the input and then rewrites exactly n elements. The algorithm takes $O(n)$ time because each element is processed a constant number of times. Thus the time complexity is $O(n)$.

7. Tests

- Test 1: $[2, 0, 2, 1, 1, 0] \rightarrow [0, 0, 1, 1, 2, 2]$.
- Test 2: $[0, 1, 2] \rightarrow [0, 1, 2]$.
- Edge case: $[] \rightarrow []$.



```
=====
ASSIGNMENT #01
=====
1. Merge Overlapping Intervals
2. Sort 0, 1, and 2
3. Largest Number
4. Group Anagrams
5. Meeting Rooms
6. K Closest Points
7. K Most Frequent Values
8. Merge K Sorted Arrays
9. Kth Largest Element
10. Count Inversions

Enter the problem number: 2
Counting solution: [0, 0, 1, 1, 2, 2]
Dutch National Flag solution: [0, 0, 1, 1, 2, 2]
(base) tapneel@tapneel-MacBook-Air: Downloads %
```

Figure 2: Problem 2 Test Case 1

3 Problem 3: Largest Number

Goal: Rearrange nonnegative integers so that their concatenation forms the largest possible number.

Input: an array of nonnegative integers.

Output: A string representing the largest concatenated number.

i. Design the Solution

Convert each integer to a string. For two strings a and b , place a before b when $ab > ba$.

ii. Pseudocode

Input: A vector of nonnegative integers, Values

Output: A string forming the largest possible number

```

1: if Values is empty then
2:     return an empty string
3: end if
4: Convert every Value in Values to a string
5: Sort the strings so a comes before b when  $a + b > b + a$ 
6: if the first string is "0" then
7:     return "0"
8: end if
9: Result <- empty string
10: for each Number in sorted strings do
11:     Result <- Result + Number
12: end for
13: return Result

```

iii. Correctness

Sorting all strings according to this rule places each pair in the order that maximizes the combined result. Concatenating the sorted strings therefore gives the largest possible number.

iv. Complexity

Sorting requires $O(n \log n)$ comparisons, with string concatenation/comparison depending on digit lengths. The space complexity is $O(L)$ because we are creating string versions of all the numbers, so it depends on the total number of digits, L .

v. Tests

- Test 1: $[3, 30, 34, 5, 9] \rightarrow 9534330$.
- Test 2: $[10, 2] \rightarrow 210$.
- Edge case: $[0, 0, 0] \rightarrow 0$.

```

=====
ASSIGNMENT #01
=====
1. Merge Overlapping Intervals
2. Sort 0, 1, and 2
3. Largest Number
4. Group Anagrams
5. Meeting Rooms
6. K Closest Points
7. K Most Frequent Values
8. Merge K Sorted Arrays
9. Kth Largest Element
10. Count Inversions

Enter the problem number: 3
Largest number: 93921710
(base) tapuwa@tapuwas-MacBook-Air Downloads % /Users/tapuwa/Downloads/Assignment01_TapuwaKabaira

```

Figure 3: Problem 3 Test Case 1

4 Problem 4: Group Anagrams

Goal: Group strings that contain the same characters with the same frequencies.

Input: An array of strings.

Output: An array of groups. Words within each group retain their original input order.

i. Design the Solution

For each word, sort its characters to create a canonical key. All anagrams have the same key. Use a hash map from the key to a vector of original words. Appending words to each group preserves their input order.

ii. Pseudocode

Input: A vector of strings, Words

Output: Groups of anagrams

```
1: Create an empty map, Groups
2: for each Word in Words do
3:     Key <- Sort the characters in Word
4:     Add Word to Groups[Key]
5: end for
6: Result <- empty list
7: for each Group in Groups do
8:     Add Group to Result
9: end for
10: return Result
```

iii. Correctness

Two words are anagrams exactly when their sorted character sequences are identical. Thus all words with the same key are placed in the same group.

iv. Complexity

If there are n words and the maximum word length is m , sorting each word costs $O(m \log m)$, giving $O(nm \log m)$ in the simple bound. Hash-map storage and the output require $O(nm)$ character storage.

v. Tests

- Test 1: [eat, tea, tan, ate, nat, bat] produces groups [eat, tea, ate], [tan, nat], [bat] in any group order.
- Test 2: [abc, bca, cab, dog] produces [abc, bca, cab] and [dog].

- Edge case: `[]` produces `[]`.

```

=====
ASSIGNMENT #01
=====
1. Merge Overlapping Intervals
2. Sort 0, 1, and 2
3. Largest Number
4. Group Anagrams
5. Meeting Rooms
6. K Closest Points
7. K Most Frequent Values
8. Merge K Sorted Arrays
9. Kth Largest Element
10. Count Inversions

Enter the problem number: 4
Anagram Groups:
[ bat ]
[ eat tea ate ]
[ tan nat ]
(base) tapuwa@Tapuwas-MacBook-Air: Downloads % /Users/tapuwa/Downloads/Assignment01_TapuwaKabaira

```

Figure 4: Problem 4 Test Case 1

5 Problem 5: Meeting Rooms

Goal: Determine whether one person can attend every meeting.

Input: Meeting intervals $[start, end]$.

Output: True if no meetings overlap; otherwise false.

Important to note: A meeting starting exactly when another ends is allowed.

i. Design the Solution

Sort meetings by starting time. Then compare each meeting's start with the previous meeting's end. An overlap exists only when the new start is strictly less than the previous end.

ii. Pseudocode

Input: An array of meeting intervals, `Meetings`

Output: True if all meetings can be attended; otherwise False

```

1: if Meetings is empty then
2:     return True
3: end if
4: Sort Meetings by increasing start time
5: for i <- 1 to SizeOf(Meetings) - 1 do
6:     if Meetings[i].start < Meetings[i - 1].end then
7:         return False
8:     end if
9: end for
10: return True

```

iii. Correctness

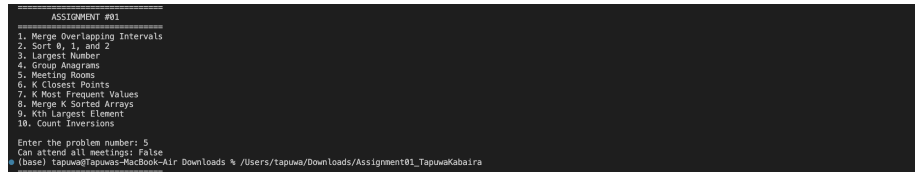
After sorting, the only possible conflict for a meeting is with the immediately preceding meeting. If its start is before the previous end, the two overlap. If no such conflict is found, all meetings can be attended.

iv. Complexity

Sorting takes $O(n \log n)$ time and scanning takes $O(n)$, so the total time is $O(n \log n)$. The space complexity is $O(n)$.

v. Tests

- Test 1: $[[0, 30], [5, 10], [15, 20]] \rightarrow false$.
- Test 2: $[[7, 10], [10, 12]] \rightarrow true$.
- Edge case: $[] \rightarrow true$.



```
=====
ASSIGNMENT #01
=====
1. Merge Overlapping Intervals
2. Sort 0, 1, and 2
3. Largest Number
4. Group Anagrams
5. Meeting Rooms
6. K Closest Points
7. K Most Frequent Values
8. Merge K Sorted Arrays
9. Kth Largest Element
10. Count Inversions

Enter the problem number: 5
Can attend all meetings: False
(base) tapuwa@Tapuwa-MacBook-Air Downloads % /Users/tapuwa/Downloads/Assignment01_TapuwaKabira
```

Figure 5: Problem 5 Test Case 1

6 Problem 6: K Closest Points

Goal: Return the k points closest to the origin.

Input: An array of points (x, y) and integer k .

Output: The k closest points.

i. Design the Solution

The sorting approach computes squared distance and sorts every point. The heap approach keeps only the k closest points seen so far in a max heap, removing the farthest whenever the heap exceeds size k .

ii. Pseudocode – Sorting

K Closest Points Using Sorting

Input: A vector of points, Points, and integer k

Output: The k points closest to the origin

```

1: for each Point in Points do
2:   Point.distance <- Point.x^2 + Point.y^2
3: end for
4: Sort Points by increasing distance
5: Result <- first k points in Points
6: return Result

```

iii. Pseudocode – Heap

Using Heap

Input: A vector of points, Points, and integer k

Output: The k points closest to the origin

```

1: Create an empty max heap, Heap
2: for each Point in Points do
3:   Distance <- Point.x^2 + Point.y^2
4:   Insert Point into Heap using Distance
5:   if SizeOf(Heap) > k then
6:     Remove the point with the largest distance
7:   end if
8: end for
9: Result <- all points in Heap
10: return Result

```

iv. Correctness

The sorting method places every point in ascending distance order, so its first k points are the required points. The heap method maintains the invariant that the heap contains the k closest points processed so far. If a new point is closer, it can replace the current farthest point.

v. Complexity

Sorting takes $O(n \log n)$ time and $O(n)$ space in the implementation. The heap method takes $O(n \log k)$ time and $O(k)$ auxiliary space.

vi. Tests

- Test 1: Points $[(1,3),(-2,2),(5,8),(0,1)]$, $k = 2$ gives $[(0,1),(-2,2)]$ up to output ordering.
- Test 2: Points $[(3,3),(5,-1),(-2,4)]$, $k = 1$ gives $[(3,3)]$.
- Edge case: $k = 0$ returns $[]$.

```
=====
ASSIGNMENT #01
1. Merge Overlapping Intervals
2. Sort 0, 1, and 2
3. Largest Number
4. Group Anagrams
5. Meeting Rooms
6. K Closest Points
7. K Most Frequent Values
8. Merge K Sorted Arrays
9. Kth Largest Element
10. Count Inversions

Enter the problem number: 6
Sorting solution:
[0, 1, 2, 2]
Heap solution:
[-2, 2] [0, 1]
(base) tapuwa@Tapuwa-MacBook-Air Downloads % /Users/tapuwa/Downloads/Assignment01_TapuwaKabaira
```

Figure 6: Problem 6 Test Case 1

7 Problem 7: K Most Frequent Values

Goal: Return the k most frequent integer values. If frequencies tie, the smaller value comes first.

Input: Integer array and integer k .

Output: The k most frequent values in the required tie order.

i. Design the Solution

Use a hash map to count each value. Convert the map entries into a list and sort by decreasing frequency. If frequencies are equal, sort by increasing value.

ii. Pseudocode

Algorithm 7 K Most Frequent Values

Input: An integer array, Values, and integer k

Output: The k most frequent values

```
1: Create an empty map, Frequency
2: for each Value in Values do
3:     Frequency[Value] <- Frequency[Value] + 1
4: end for
5: Create a list of all unique values
6: Sort the list by decreasing frequency
7: If frequencies are equal, sort by increasing value
8: Result <- first  $k$  values from the sorted list
9: return Result
```

iii. Correctness

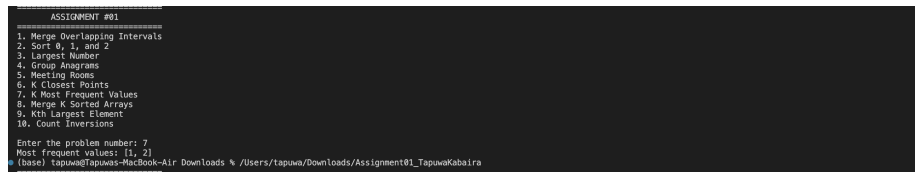
The hash map records the exact frequency of every value. Sorting the pairs by decreasing frequency places the most frequent values first. The tie rule then places smaller values first whenever two frequencies are equal.

iv. Complexity

Counting is $O(n)$ time. If u is the number of distinct values, sorting takes $O(u \log u)$. Total expected time is $O(n + u \log u)$ and space is $O(u)$.

v. Tests

- Test 1: $[1,1,1,2,2,3]$, $k = 2 \rightarrow [1, 2]$.
- Test 2: $[4,4,5,5,3,3]$, $k = 2 \rightarrow [3, 4]$ because the frequencies tie.
- Edge case: $[], k = 2 \rightarrow []$.



```
ASSIGNMENT #01
1. Merge Overlapping Intervals
2. Sort 0, 1, and 2
3. Largest Number
4. Group Anagrams
5. Meeting Rooms
6. K Closest Points
7. K Most Frequent Values
8. Merge K Sorted Arrays
9. Kth Largest Element
10. Count Inversions

Enter the problem number: 7
Most frequent values: [1, 2]
(base) tapuwa@Tapuwa-MacBook-Air Downloads % /Users/tapuwa/Downloads/Assignment01_TapuaKabira
```

Figure 7: Problem 7 Test Case 1

8 Problem 8: Merge K Sorted Arrays

Goal: Merge k individually sorted arrays into one sorted array without repeatedly sorting the complete output.

Input: k sorted integer arrays.

Output: One sorted array containing every input element.

i. Design the Solution

Put the first element of every nonempty array into a min heap. Repeatedly remove the smallest element and insert the next element from the same array.

ii. Pseudocode

Input: K sorted arrays, Arrays

Output: One sorted array containing all values

```
1: Create an empty min heap, Heap
2: for each Array in Arrays do
3:     if Array is not empty then
4:         Insert the first value of Array into Heap
5:     end if
6: end for
7: Result <- empty list
```

```

8: while Heap is not empty do
9:   Smallest <- Remove the smallest value from Heap
10:  Add Smallest to Result
11:  if the array containing Smallest has another value then
12:    Insert the next value from that array into Heap
13:  end if
14: end while
15: return Result

```

iii. Correctness

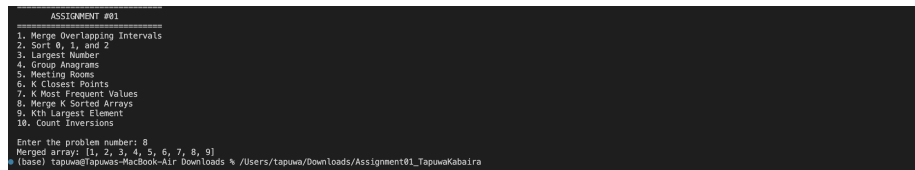
The heap always contains the smallest not-yet-output element from each nonempty array. Therefore its minimum is the smallest remaining element across all arrays. After outputting it, the next element from that same array is inserted.

iv. Complexity

If N is the total number of elements and k the number of arrays. Each element enters and leaves the heap once, so the time complexity is $O(N \log k)$.

v. Tests

- Test 1: $[[1,4,7],[2,5,8],[3,6,9]] \rightarrow [1,2,3,4,5,6,7,8,9]$.
- Test 2: $[[],[1,3],[2],[]] \rightarrow [1,2,3]$.
- Edge case: $[] \rightarrow []$.



```

ASSIGNMENT #01
1. Merge Overlapping Intervals
2. Sort 0, 1, and 2
3. Largest Number
4. Group Anagrams
5. Meeting Rooms
6. K Closest Points
7. K Most Frequent Values
8. Merge K Sorted Arrays
9. Kth Largest Element
10. Count Inversions

Enter the problem number: 8
Merged array: [1, 2, 3, 4, 5, 6, 7, 8, 9]
(base) tapuwa@tapuwas-MacBook-Air: Downloads % /Users/tapuwa/Downloads/Assignment01_TapuwaKabaira

```

Figure 8: Problem 8 Test Case 1

9 Problem 9: Kth Largest Element

Goal: Return the k th-largest element.

Input: An unsorted integer array and valid integer k .

Output: The k th-largest integer.

i. Design the Solution

Maintain a min heap containing the largest k values seen so far. If the heap grows beyond k , remove its smallest value. At the end, the smallest value in the heap is the k th-largest value.

ii. Pseudocode

Input: An integer array Values and integer k

Output: The k th-largest value

```
1: Create an empty min heap, Heap
2: for each Value in Values do
3:     Insert Value into Heap
4:     if SizeOf(Heap) > k then
5:         Remove the smallest value from Heap
6:     end if
7: end for
8: return the smallest value in Heap
```

iii. Correctness

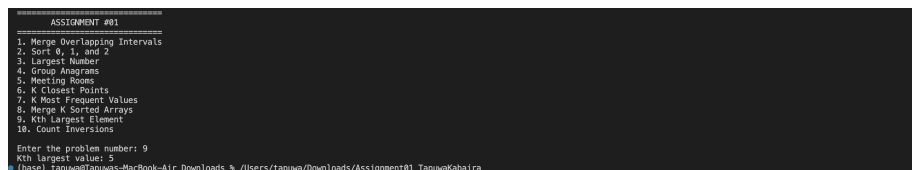
Values smaller than the current top k can be discarded. After all values are processed, the smallest member of the top k values is exactly the k th-largest value. Because every occurrence is inserted separately, duplicates count as separate positions.

iv. Complexity

Each of n values is inserted into a heap of size at most k , giving $O(n \log k)$ time.

v. Tests

- Test 1: $[3,2,1,5,6,4]$, $k = 2 \rightarrow 5$.
- Test 2: $[3,2,3,1,2,4,5,5,6]$, $k = 4 \rightarrow 4$.
- Edge case: $[7]$, $k = 1 \rightarrow 7$.



```
=====
ASSIGNMENT #01
1. Merge Overlapping Intervals
2. Sort 0, 1, and 2
3. Largest Number
4. Group Anagrams
5. Meeting Rooms
6. K Closest Points
7. K Most Frequent Values
8. Merge K Sorted Arrays
9. Kth Largest Element
10. Count Inversions

Enter the problem number: 9
kth largest value: 5
(base) tapua@tapuas-MacBook-Air Downloads % /Users/tapua/Downloads/Assignment01_TapuaKabira
```

Figure 9: Problem 9 Test Case 1

10 Problem 10: Count Inversions

Goal: Count pairs (i, j) such that $i < j$ and $Values[i] > Values[j]$.

Input: An unsorted integer array.

Output: The number of inversions as a 64-bit integer.

Requirement: The primary solution must run in $O(n \log n)$ time.

i. Design the Solution

During merging, if the next element from the right half is smaller than the next element from the left half, then it forms an inversion with every remaining element in the left half. If the left pointer is i and the midpoint is m , the number of new inversions is $m - i + 1$.

ii. Pseudocode

Input: An integer array `Values`

Output: Number of inversions

```
1: if SizeOf(Values) < 2 then
2:     return 0
3: end if
4: Divide Values into left and right halves
5: LeftCount <- Count inversions in left half
6: RightCount <- Count inversions in right half
7: Merge the two sorted halves
8: During merge, if Left[i] > Right[j] then
9:     Add the number of remaining elements in Left to the count
10: end if
11: return LeftCount + RightCount + MergeCountState an invariant or exchange argument
```

iii. Correctness

Merge Sort recursively counts every inversion entirely inside the left half and every inversion entirely inside the right half. When a right-half value is smaller than the current left-half value, it is smaller than every remaining left-half value, so exactly all of those pairs are inversions.

iv. Complexity

The recurrence is $T(n) = 2T(n/2) + O(n)$, giving $O(n \log n)$ time complexity. The temporary array requires $O(n)$ space.

v. Tests

- Required example: $[2,4,1,3,5] \rightarrow 3$.
- Additional test: $[5,4,3,2,1] \rightarrow 10$.
- Edge case: $[1,2,3,4] \rightarrow 0$.

```
=====
ASSIGNMENT #01
=====
1. Merge Overlapping Intervals
2. Sort 0, 1, and 2
3. Largest Number
4. Group Anagrams
5. Meeting Rooms
6. K Closest Points
7. K Most Frequent Values
8. Merge K Sorted Arrays
9. Kth Largest Element
10. Count Inversions

Enter the problem number: 10
Number of Inversions: 3
(base) tapuwa@Tapuwa-MacBook-Air Downloads % /Users/tapuwa/Downloads/Assignment01_TapuwaKabira
```

Figure 10: Problem 10 Test Case 1